

PESS-Project1

小组成员：程传哲（学号：23040032008）

袁东霖（学号：23020006170）

张仕达（学号：23010022094）

郭晓伟（学号：23060021010）

完成日期： 2026 年 3 月 22 日

实验目录

PESS-Project1	1
一、 实验目的	3
二、 实验介绍	3
三、 Perf 定位性能瓶颈	3
四、 实验代码构思与伪代码	5
五、 优化编码	6
六、 测试样例构思	7
七、 代码实验截图	8
八、 附录	10

一、实验目的

1. 使用 perf 工具，定位性能瓶颈，指导优化方向；
2. 字级并行：把 word 当作多个可并行的处理单元 bit；
3. 在保证程序正确运行的基础上，进行程序性能评估

二、实验介绍

project1约束条件：

- 不要执行多线程并行化；
- 以标准C代码为例，不能使用内联汇编指令；
 - 也不要试图借助编译器内部函数或使用汇编或内部函数的库达到相同的效果；

代码结构

- everybit/ 目录
 - main.c 、 test.c
 - 功能：计时、测试系统
 - 要求：**不要删改！不要删改！不要删改！**
 - tests/ 目录
 - default
 - 课程提供的一些测试用例
 - mytests/
 - Task2: 需手动编写测试用例，以提高测试覆盖率
 - 目标: 尽可能考虑极端情况、边界条件
 - 注意：团队内设计的测试用例会用于团队间测试（A组的测试用例用于B组的测试，若bug逃逸率过高将影响A组分数）。
 - bitarray.c 、 bitarray.h
 - 功能：实现了使用最小内存的情况下分配、访问、处理大型位字符串的函数
 - 内存中的 bit 以字节的形式存在，可通过 bitarray_get() 和 bitarray_set() 函数单独访问每个 bit
 - Task1: 优化 bitarray_rotate() 函数
 - 要求：不能调用 bitarray_new() 函数
 - 验证：bit数组调用 优化后的bitarray_rotate() 后的内容与调用 未优化的bitarray_rotate() 内容一致即可
 - 提示：可以在堆栈、BSS段分配小缓冲区，如全局数组； bitarray_get_bit_sz() 和 bitarray_get() 函数可用于访问bit数组内容

本实验围绕位数组的循环旋转操作展开，目标是在严格约束条件下完成程序的正确性验证与性能优化。实验给定了一个基于 C 语言实现的位数组项目，要求在不改变主测试框架的前提下，对核心函数 bitarray_rotate() 进行分析与优化，并补充测试样例以提高程序的正确性验证覆盖率。

三、Perf 定位性能瓶颈

为了避免盲目优化，在修改核心代码之前，先使用 perf 工具对程序进行性能分析，定位程序运行过程中的主要热点函数，并据此确定后续优化方向。

选择对 ./everybit -l 进行分析。原因是 -l 对应的大规模性能测试最能体现算法复杂度带来的运行时间差异，因此更适合用于定位性能瓶颈。

```
----- RESULTS -----
Tier 0 (~0B) completed in 0.000003s
Tier 1 (~0B) completed in 0.000001s
Tier 2 (~1B) completed in 0.000001s
Tier 3 (~1B) completed in 0.000002s
Tier 4 (~2B) completed in 0.000004s
Tier 5 (~4B) completed in 0.000009s
Tier 6 (~6B) completed in 0.000021s
Tier 7 (~11B) completed in 0.000052s
Tier 8 (~18B) completed in 0.000129s
Tier 9 (~29B) completed in 0.000344s
Tier 10 (~47B) completed in 0.001122s
Tier 11 (~76B) completed in 0.002422s
Tier 12 (~123B) completed in 0.007540s
Tier 13 (~199B) completed in 0.014287s
Tier 14 (~323B) completed in 0.020945s
Tier 15 (~522B) completed in 0.057602s
Tier 16 (~845B) completed in 0.151039s
Tier 17 (~1KB) completed in 0.409942s
Tier 18 (~2KB) exceeded 1.00s cutoff with time 1.099647s
Successfully completed tier: 17
----- END RESULTS -----

Performance counter stats for './everybit -l':

      1766.64 msec task-clock:u          #    0.976 CPUs utilized
           0      context-switches:u    #    0.000 /sec
           0      cpu-migrations:u      #    0.000 /sec
          56      page-faults:u         #   31.699 /sec
<not supported> cycles:u
<not supported> instructions:u
<not supported> branches:u
<not supported> branch-misses:u

      1.810773604 seconds time elapsed

      1.767510000 seconds user
      0.000000000 seconds sys
```

从实验结果来看，在 `perf stat ./everybit -l` 条件下，程序最终 Successfully completed tier: 17，并在更高级测试中超过时间阈值。与此同时，`perf` 给出了如下信息：task-clock 约为 1766.64 ms，page-faults 为 56，总运行时间约为 1.8108 s。这一点说明了在输入规模增大的情况下，程序执行时间增长显著，存在很大优化空间。

由于实验环境是 Wsl2，底层对部分硬件 PMU 事件支持有限，cycles、instructions、branches 等硬件计数器也显示为 <not supported>。不过并不影响本实验使用 `perf` 对程序进行整体性能观测，也不影响结合源码对热点路径进行定位和分析。

```
ccz@chengchuanzhe:~/projects_1/bg/everybit$ /usr/lib/linux-tools/6.8.0-106-generic/perf report --stdio
# To display the perf.data header info, please use --header/--header-only options.
#
#
# Total Lost Samples: 0
#
# Samples: 7K of event 'task-clock:upph'
# Event count (approx.): 1855750000
#
# Overhead Command Shared Object Symbol
# .....
#
# 48.34% everybit everybit [.] bitarray_set
# 21.33% everybit everybit [.] bitarray_get
# 18.51% everybit everybit [.] bitmask
# 11.81% everybit everybit [.] bitarray_rotate_left_one
# 0.01% everybit libc.so.6 [.] __printf_fp_buffer_1.isra.0
```

程序中真正决定性能的函数是 `bitarray_rotate()`。在原始实现中，`bitarray_rotate()` 会先将右旋统一转换为左旋，然后调用 `bitarray_rotate_left()` 完成实际操作。而 `bitarray_rotate_left()` 的实现方式是：如果需要旋转 `k` 位，就重复调用 `bitarray_rotate_left_one()` 共 `k` 次；而每执行一次 `bitarray_rotate_left_one()`，都需要对目标区间中的 bit 逐位访问、逐位搬移，并频繁调用 `bitarray_get()` 和 `bitarray_set()` 进行位级

读写。

通过事件记录可以发现 `bitarray_get()` 和 `bitarray_set()` 的调用占比几乎达到的 70%，实验将从此得出的代码的本质问题它把一次整体旋转拆解成了大量重复的单 `bit` 移动操作

基于上述分析，后续采用了基于区间翻转的旋转方法，将原先多次执行单 `bit` 搬移的做法改写为整体区间操作，从而显著降低算法复杂度，并提高程序在大规模输入下的性能表现。

四、实验代码构思与伪代码

4.1 项目代码整体理解

项目围绕一个位数组数据结构展开。与普通数组不同，位数组不是把每一位单独存为一个独立变量，而是将多个 `bit` 紧凑地压缩到字节数组中存储。这种设计的优点是节省内存，但缺点是每次读取或修改某一位时，都需要通过位运算计算它所在的字节位置和字节内偏移，因此操作粒度较细，容易在大量访问时形成性能瓶颈。

4.2 原始实现思路分析

先保存区间最左边的 `bit`；
将后面的每个 `bit` 向左移动一格；
再把最开始保存的那个 `bit` 放到区间最右端。

算法整体复杂度接近 $O(n * k)$ 。当区间长度和旋转位数都较大时，会产生大量重复 `bit` 访问与赋值操作

4.3 优化后的代码构思

实验约束：不使用多线程；不使用 SIMD 或内联汇编；不调用 `bitarray_new()` 新建额外位数组；在原有 `bitarray` 上原地完成旋转。

核心思想

假设一个区间长度为 `n`，需要左旋 `k` 位。

可以把区间分成两部分：

前半部分 A：前 `k` 位

后半部分 B：后 `n-k` 位

左旋后的目标结果是：

`A B -> B A`

这个变换可以通过三次翻转完成：

翻转 A，翻转 B，翻转整个区间 AB

这样就能在原数组上得到旋转后的结果。

这种方法的优点是：不需要额外大块缓存空间；每次翻转只需双指针向中间交换；总共只需对区间做 3 次线性扫描。

4.4 优化后核心算法伪代码

左旋函数伪代码

函数 `ROTATE_LEFT(bitarray, offset, length, left_amount):`

如果 `length <= 1` 或 `left_amount == 0` 或 `left_amount == length`:

直接返回

```
REVERSE_SUBARRAY(bitarray, offset, left_amount)
REVERSE_SUBARRAY(bitarray, offset + left_amount, length - left_amount)
REVERSE_SUBARRAY(bitarray, offset, length)
```

区间翻转函数伪代码

函数 REVERSE_SUBARRAY(bitarray, offset, length):

如果 length <= 1:

直接返回

left = offset

right = offset + length - 1

当 left < right 时重复:

left_bit = GET(bitarray, left)

right_bit = GET(bitarray, right)

SET(bitarray, left, right_bit)

SET(bitarray, right, left_bit)

left = left + 1

right = right - 1

五、 优化编码

```
void bitarray_rotate(bitarray_t* const bitarray,
                    const size_t bit_offset,
                    const size_t bit_length,
                    const ssize_t bit_right_amount) {
    assert(bit_offset + bit_length <= bitarray->bit_sz);

    if (bit_length == 0) {
        return;
    }

    // Convert a rotate left or right to a left rotate only, and eliminate
    // multiple full rotations.
    bitarray_rotate_left(bitarray, bit_offset, bit_length,
                        modulo(-bit_right_amount, bit_length));
}

static void bitarray_rotate_left(bitarray_t* const bitarray,
                                const size_t bit_offset,
                                const size_t bit_length,
                                const size_t bit_left_amount) {
    if (bit_length <= 1 || bit_left_amount == 0 || bit_left_amount == bit_length) {
        return;
    }

    bitarray_reverse_subarray(bitarray, bit_offset, bit_left_amount);
    bitarray_reverse_subarray(bitarray, bit_offset + bit_left_amount,
                            bit_length - bit_left_amount);
    bitarray_reverse_subarray(bitarray, bit_offset, bit_length);
}
```

```
static void bitarray_reverse_subarray(bitarray_t* const bitarray,
                                     const size_t bit_offset,
                                     const size_t bit_length) {
    if (bit_length <= 1) {
        return;
    }

    size_t left = bit_offset;
    size_t right = bit_offset + bit_length - 1;
    while (left < right) {
        const bool left_bit = bitarray_get(bitarray, left);
        const bool right_bit = bitarray_get(bitarray, right);
        bitarray_set(bitarray, left, right_bit);
        bitarray_set(bitarray, right, left_bit);
        left++;
        right--;
    }
}
```

六、测试样例构思

(1) 极小规模边界

如长度为 1 或 2 的 bit 串。

这类测试用于验证当区间极短时，程序是否会出现越界、无意义翻转或错误修改。

(2) 旋转量为 0、等于长度、大于长度

这类测试主要验证模运算处理是否正确。

例如长度为 8 的区间，旋转 8 位应等价于不旋转，旋转 9 位应等价于旋转 1 位。

(3) 负数旋转

接口允许传入负数，因此需要验证程序是否能正确把负数旋转量转为等价方向的旋转。

(4) 子区间旋转

项目并不是只支持整个 bit 串旋转，而是支持任意 `offset + length` 的子区间旋转。

因此必须测试：

区间内的 bit 是否被正确旋转；

区间外的 bit 是否保持不变。

(5) 跨字节边界测试

由于 bitarray 底层按字节压缩存储，如果待旋转区间跨越多个字节，就更容易暴露位运算和边界处理问题。因此设计了 16 位、20 位等更长字符串测试，以检查跨字节场景下的正确性。

(6) 连续多次旋转

部分测试不仅做一次旋转，而是在同一个 bit 串上连续执行两次旋转。

这样可以检验代码在连续操作时是否依然稳定，而不是只在单次调用下侥幸正确。

七、 代码实验截图

```
ccz@chengchuanzhe: ~/projex  X + v

inflating: everybit/bitarray.h
inflating: everybit/clint.py
inflating: everybit/ktiming.c
inflating: everybit/ktiming.h
inflating: everybit/main.c
inflating: everybit/Makefile
inflating: everybit/tests.c
inflating: everybit/tests.h
  creating: everybit/tests/
inflating: everybit/tests/default
inflating: everybit/tests/mytests
inflating: everybit/bitarray.c.bak
extracting: everybit/.buildmode
inflating: everybit/bitarray.o
inflating: everybit/ktiming.o
inflating: everybit/main.o
inflating: everybit/tests.o
inflating: everybit/everybit
ccz@chengchuanzhe:~/projects$ ls
everybit
ccz@chengchuanzhe:~/projects$ find ~/projects -type d | grep everybit
/home/ccz/projects/everybit
/home/ccz/projects/everybit/tests
ccz@chengchuanzhe:~/projects$ cd everybit
ccz@chengchuanzhe:~/projects/everybit$ sed -i 's/\r$//' tests/default tests/mytests 2>/dev/null || true
ccz@chengchuanzhe:~/projects/everybit$ make clean
make: Warning: File 'Makefile' has modification time 19359 s in the future
rm -f everybit *.o .buildmode *.gcov *.gcnv *.gcda
make: warning: Clock skew detected. Your build may be incomplete.
ccz@chengchuanzhe:~/projects/everybit$ |

ccz@chengchuanzhe: ~/projex  X + v

ccz@chengchuanzhe:~/projects$ ls
everybit
ccz@chengchuanzhe:~/projects$ find ~/projects -type d | grep everybit
/home/ccz/projects/everybit
/home/ccz/projects/everybit/tests
ccz@chengchuanzhe:~/projects$ cd everybit
ccz@chengchuanzhe:~/projects/everybit$ sed -i 's/\r$//' tests/default tests/mytests 2>/dev/null || true
ccz@chengchuanzhe:~/projects/everybit$ make clean
make: Warning: File 'Makefile' has modification time 19359 s in the future
rm -f everybit *.o .buildmode *.gcov *.gcnv *.gcda
make: warning: Clock skew detected. Your build may be incomplete.
ccz@chengchuanzhe:~/projects/everybit$ find . -type f -exec touch {} +
ccz@chengchuanzhe:~/projects/everybit$ make DEBUG=1 EXTRA_LDFLAGS="-fuse-ld=lld"
clang -std=c99 -Wall -m64 -g -O0 -o bitarray.o -c bitarray.c
bitarray.c:238:13: warning: unused function 'bitarray_rotate_left_one' [-Wunused-function]
    238 | static void bitarray_rotate_left_one(bitarray_t* const bitarray,
        |
1 warning generated.
clang -std=c99 -Wall -m64 -g -O0 -o ktiming.o -c ktiming.c
clang -std=c99 -Wall -m64 -g -O0 -o main.o -c main.c
clang -std=c99 -Wall -m64 -g -O0 -o tests.o -c tests.c
tests.c:267:14: warning: variable length array folded to constant array as an extension [-Wgnu-folding-constant]
    267 | const double fibs[FIB_SIZE] = {1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, 233, 377, 610, 987, 1597, 2584, 4181, 676
      |
5, 10946, 17711, 28657, 46368, 75025, 121393, 196418, 317811, 514229, 832040, 1346269, 2178309, 3524578, 5702887, 922746
5, 14930352, 24157817, 39088169, 63245986, 102334155, 165580141, 267914296, 433494437, 701408733, 1134903170, 1836311903
, 2971215073, 4807526976, 7778742049, 12586269025, 20365011074, 32951280099, 53316291173, 86267571272};
      |
1 warning generated.
clang bitarray.o ktiming.o main.o tests.o -flto -fuse-ld=gold -lrt -fuse-ld=lld -o everybit
ccz@chengchuanzhe:~/projects/everybit$ |
```



```
ccz@chengchuanzhe: ~/projex X + v
1 warning generated.
clang bitarray.o ktiming.o main.o tests.o -flto -fuse-ld=gold -lrt -fuse-ld=lld -o everybit
ccz@chengchuanzhe:~/projects/everybit$ ls -l everybit
-rwxr-xr-x 1 ccz ccz 31080 Mar 19 00:30 everybit
ccz@chengchuanzhe:~/projects/everybit$ ./everybit
./everybit -t tests/default
./everybit -t tests/mytests
usage: ./everybit
        -s Run a sample small (0.01s) rotation operation
        -m Run a sample medium (0.1s) rotation operation
        -l Run a sample large (1s) rotation operation
           (note: the provided [-s/m/l] options only test performance and NOT correctness.)
        -t tests/default      Run all tests in the testfile tests/default
        -n 1 -t tests/default Run test 1 in the testfile tests/default
Testing file tests/default.

Running test #0...
--> tests/default at line 34: PASS

Running test #1...
--> tests/default at line 41: PASS

Running test #2...
--> tests/default at line 49: PASS
--> tests/default at line 52: PASS
--> tests/default at line 55: PASS
--> tests/default at line 58: PASS
--> tests/default at line 61: PASS
Done testing file tests/default.
10010101001011010000101Testing file tests/mytests.
```

```
ccz@chengchuanzhe: ~/projex X + v

Running test #0...
--> tests/mytests at line 26: PASS

Running test #1...
--> tests/mytests at line 31: PASS

Running test #2...
--> tests/mytests at line 36: PASS

Running test #3...
--> tests/mytests at line 41: PASS

Running test #4...
--> tests/mytests at line 46: PASS

Running test #5...
--> tests/mytests at line 51: PASS

Running test #6...
--> tests/mytests at line 56: PASS

Running test #7...
--> tests/mytests at line 61: PASS

Running test #8...
--> tests/mytests at line 66: PASS

Running test #9...
--> tests/mytests at line 71: PASS
```

```
ccz@chengchuanzhe: ~/projex X + v
Done testing file tests/mytests.
0101010101100101010010101011000011100000101010010101011001100110011001000000000000000ccz@chengchuanzhe:~/projects/everybit $ ./everybit --s101010:ccz@chengchuanzhe:~/projects/everybit -l
----- RESULTS -----
Tier 0 (#0B) completed in 0.000002s
Tier 1 (#0B) completed in 0.000001s
Tier 2 (#1B) completed in 0.000001s
Tier 3 (#1B) completed in 0.000001s
Tier 4 (#2B) completed in 0.000001s
Tier 5 (#4B) completed in 0.000002s
Tier 6 (#6B) completed in 0.000003s
Tier 7 (#11B) completed in 0.000006s
Tier 8 (#18B) completed in 0.000006s
Tier 9 (#29B) completed in 0.000009s
Tier 10 (#47B) completed in 0.000018s
Tier 11 (#76B) completed in 0.000023s
Tier 12 (#123B) completed in 0.000036s
Tier 13 (#199B) completed in 0.000050s
Tier 14 (#323B) completed in 0.000066s
Tier 15 (#522B) completed in 0.000276s
Tier 16 (#840B) completed in 0.000268s
Tier 17 (#140B) completed in 0.000394s
Tier 18 (#240B) completed in 0.000633s
Tier 19 (#390B) completed in 0.001068s
Tier 20 (#640B) completed in 0.001839s
Tier 21 (#990B) completed in 0.003426s
Tier 22 (#140B) completed in 0.005019s
Tier 23 (#230B) completed in 0.007926s
Tier 24 (#380B) completed in 0.009375s
Tier 25 (#620B) exceeded 0.01s cutoff with time 0.018132s
Successfully completed tier: 24
----- END RESULTS -----
----- RESULTS -----
Tier 0 (#0B) completed in 0.000001s
Tier 1 (#0B) completed in 0.000001s
Tier 2 (#1B) completed in 0.000001s
Tier 3 (#1B) completed in 0.000001s
Tier 4 (#2B) completed in 0.000001s
Tier 5 (#4B) completed in 0.000002s
Tier 6 (#6B) completed in 0.000002s
Tier 7 (#11B) completed in 0.000003s
Tier 8 (#18B) completed in 0.000005s
Tier 9 (#29B) completed in 0.000007s
Tier 10 (#47B) completed in 0.000012s
Tier 11 (#76B) completed in 0.000018s
Tier 12 (#123B) completed in 0.000039s
Tier 13 (#199B) completed in 0.000047s
Tier 14 (#323B) completed in 0.000078s
Tier 15 (#522B) completed in 0.000108s
```

```
ccz@chengchuanzhe:~/projex  X  +  v
Successfully completed tier: 24
----- END RESULTS -----
----- RESULTS -----
Tier 0 (=8B) completed in 0.000001s
Tier 1 (=8B) completed in 0.000001s
Tier 2 (=1B) completed in 0.000001s
Tier 3 (=1B) completed in 0.000001s
Tier 4 (=2B) completed in 0.000001s
Tier 5 (=4B) completed in 0.000002s
Tier 6 (=6B) completed in 0.000002s
Tier 7 (=11B) completed in 0.000003s
Tier 8 (=18B) completed in 0.000005s
Tier 9 (=29B) completed in 0.000007s
Tier 10 (=47B) completed in 0.000012s
Tier 11 (=76B) completed in 0.000018s
Tier 12 (=123B) completed in 0.000035s
Tier 13 (=199B) completed in 0.000047s
Tier 14 (=323B) completed in 0.000078s
Tier 15 (=522B) completed in 0.000108s
Tier 16 (=845B) completed in 0.000201s
Tier 17 (=140B) completed in 0.000346s
Tier 18 (=240B) completed in 0.000570s
Tier 19 (=390B) completed in 0.000902s
Tier 20 (=590B) completed in 0.001310s
Tier 21 (=940B) completed in 0.002133s
Tier 22 (=1400B) completed in 0.002451s
Tier 23 (=2300B) completed in 0.003621s
Tier 24 (=3800B) completed in 0.005828s
Tier 25 (=6200B) completed in 0.009384s
Tier 26 (=10100B) completed in 0.015041s
Tier 27 (=16400B) completed in 0.023687s
Tier 28 (=26500B) completed in 0.039621s
Tier 29 (=43900B) completed in 0.064399s
Tier 30 (=69600B) exceeded 0.10s cutoff with time 0.103475s
Successfully completed tier: 29
----- END RESULTS -----
----- RESULTS -----
Tier 0 (=8B) completed in 0.000001s
Tier 1 (=8B) completed in 0.000000s
Tier 2 (=1B) completed in 0.000000s
Tier 3 (=1B) completed in 0.000001s
Tier 4 (=2B) completed in 0.000001s
Tier 5 (=4B) completed in 0.000001s
Tier 6 (=6B) completed in 0.000001s
Tier 7 (=11B) completed in 0.000002s
Tier 8 (=18B) completed in 0.000003s
Tier 9 (=29B) completed in 0.000005s
Tier 10 (=47B) completed in 0.000008s
Tier 11 (=76B) completed in 0.000013s
Tier 12 (=123B) completed in 0.000020s
Tier 13 (=199B) completed in 0.000031s
Tier 14 (=323B) completed in 0.000048s
Tier 15 (=522B) completed in 0.000077s
Tier 16 (=845B) completed in 0.000129s
Tier 17 (=140B) completed in 0.000206s
Tier 18 (=240B) completed in 0.000321s
Tier 19 (=390B) completed in 0.000551s
Tier 20 (=590B) completed in 0.000840s
Tier 21 (=940B) completed in 0.001492s
Tier 22 (=1400B) completed in 0.002262s
Tier 23 (=2300B) completed in 0.003701s
Tier 24 (=3800B) completed in 0.005942s
Tier 25 (=6200B) completed in 0.009488s
Tier 26 (=10100B) completed in 0.015144s
Tier 27 (=16400B) completed in 0.024888s
Tier 28 (=26500B) completed in 0.039751s
Tier 29 (=43900B) completed in 0.064112s
Tier 30 (=69600B) completed in 0.109751s
Tier 31 (=110B) completed in 0.168633s
Tier 32 (=180B) completed in 0.261862s
Tier 33 (=290B) completed in 0.410921s
Tier 34 (=460B) completed in 0.682787s
Tier 35 (=760B) exceeded 1.00s cutoff with time 1.099655s
Successfully completed tier: 34
ccz@chengchuanzhe:~/projects/everybit$ |
```

```
ccz@chengchuanzhe:~/projex  X  +  v
Tier 27 (=16400B) completed in 0.023687s
Tier 28 (=26500B) completed in 0.039621s
Tier 29 (=43900B) completed in 0.064399s
Tier 30 (=69600B) exceeded 0.10s cutoff with time 0.103475s
Successfully completed tier: 29
----- END RESULTS -----
----- RESULTS -----
Tier 0 (=8B) completed in 0.000001s
Tier 1 (=8B) completed in 0.000000s
Tier 2 (=1B) completed in 0.000000s
Tier 3 (=1B) completed in 0.000001s
Tier 4 (=2B) completed in 0.000001s
Tier 5 (=4B) completed in 0.000001s
Tier 6 (=6B) completed in 0.000001s
Tier 7 (=11B) completed in 0.000002s
Tier 8 (=18B) completed in 0.000003s
Tier 9 (=29B) completed in 0.000005s
Tier 10 (=47B) completed in 0.000008s
Tier 11 (=76B) completed in 0.000013s
Tier 12 (=123B) completed in 0.000020s
Tier 13 (=199B) completed in 0.000031s
Tier 14 (=323B) completed in 0.000048s
Tier 15 (=522B) completed in 0.000077s
Tier 16 (=845B) completed in 0.000129s
Tier 17 (=140B) completed in 0.000206s
Tier 18 (=240B) completed in 0.000321s
Tier 19 (=390B) completed in 0.000551s
Tier 20 (=590B) completed in 0.000840s
Tier 21 (=940B) completed in 0.001492s
Tier 22 (=1400B) completed in 0.002262s
Tier 23 (=2300B) completed in 0.003701s
Tier 24 (=3800B) completed in 0.005942s
Tier 25 (=6200B) completed in 0.009488s
Tier 26 (=10100B) completed in 0.015144s
Tier 27 (=16400B) completed in 0.024888s
Tier 28 (=26500B) completed in 0.039751s
Tier 29 (=43900B) completed in 0.064112s
Tier 30 (=69600B) completed in 0.109751s
Tier 31 (=110B) completed in 0.168633s
Tier 32 (=180B) completed in 0.261862s
Tier 33 (=290B) completed in 0.410921s
Tier 34 (=460B) completed in 0.682787s
Tier 35 (=760B) exceeded 1.00s cutoff with time 1.099655s
Successfully completed tier: 34
ccz@chengchuanzhe:~/projects/everybit$ |
```

tests/default: 通过

tests/mytests: 通过

性能测试结果:

./everybit -s → tier 24

./everybit -m → tier 29

./everybit -l → tier 34

可以看到对比为优化的代码./everybit -l 提升近一倍

八、附录

具体测试样例

```
t 0
n 0
r 010
```

e 0

t 1

n 1

r 0 1 7

e 1

t 2

n 0 1

r 0 2 1

e 1 0

t 3

n 1 0

r 0 2 -1

e 0 1

t 4

n 1 0 1

r 0 3 1

e 1 1 0

t 5

n 1 0 1

r 0 3 -1

e 0 1 1

t 6

n 1 0 0 1 0 1 1 0

r 0 8 8

e 1 0 0 1 0 1 1 0

t 7

n 1 0 0 1 0 1 1 0

r 0 8 9

e 0 1 0 0 1 0 1 1

t 8

n 1 0 0 1 0 1 1 0

r 0 8 -9

e 0 0 1 0 1 1 0 1

t 9

n 1 1 1 1 0 0 0 0

r 2 4 1
e 11011000

t 10
n 11110000
r 2 4 -1
e 11100100

t 11
n 010101010
r 1 7 3
e 010110100

t 12
n 010101010
r 1 7 -3
e 001011010

t 13
n 110011001100
r 3 6 2
e 110010110100

t 14
n 110011001100
r 3 6 -2
e 110100101100

t 15
n 1000000000000001
r 1 14 5
e 1000000000000001

t 16
n 1000000000000001
r 0 16 -4
e 0000000000011000

t 17
n 10110011100011110000
r 4 12 5
e 10110111100111000000

t 18

n 101100111100011110000
r 0 20 -7
e 110001111100001011001
t 19
n 00101101101010
r 2 7 20
e 0001101111010
r 0 12 -5
e 011101000011

耗时表

设计	编码	测试样例设计	测试调试	会议
1h	1h	30min	40min	35min